并行计算

——（2）算法

讲授：鄂金龙

1.并行算法的设计基础

并行算法的定义和分类

- 并行算法的定义
  - 算法
  - 并行算法：一些可同时执行的诸进程的集合，这些进程互相作用和协调动作从而达到给定问题的求解。
- 并行算法的分类
  - 数值计算和非数值计算
  - 同步算法和异步算法
  - 分布算法
  - 确定算法和随机算法

并行算法的表达

- 描述语言
  - 可以使用类Algol、类Pascal等；
  - 在描述语言中引入并行语句。
- 并行语句示例
  - Par-do语句

```
for i=1 to n par-do
    ……
end for
```

  - for all语句

```
for all Pi, where 0≤i≤k
  ……
end for
```

并行算法的复杂性度量

- 串行算法的复杂性度量
  - 最坏情况下的复杂度(Worst-CASE Complexity)
  - 期望复杂度(Expected Complexity)
- 并行算法的几个复杂性度量指标
  - 运行时间t(n):包含计算时间和通讯时间，分别用计算时间步和选路时间步作单位。n为问题实例的输入规模。
  - 处理器数p(n)
  - 并行算法成本c(n): c(n)=t(n)p(n)
  - 总运算量W(n): 并行算法求解问题时所完成的总的操作步数。

并行算法的复杂性度量

- Brent定理

  令W(n)是某并行算法A在运行时间T(n)内所执行的运算量，则A使用p台处理器可在t(n)=O(W(n)/p+T(n))时间内执行完毕。

  - W(n)和c(n)密切相关
  - P=O(W(n)/T(n))时，W(n)和c(n)两者是渐进一致的
  - 对于任意的p，c(n)>W(n)

并行算法的同步

- 同步概念
  - 同步是在时间上强使各执行进程在某一点必须互相等待；
  - 可用软件、硬件和固件的办法来实现。
- 同步语句示例
  - 算法4.1 共享存储多处理器上求和算法

    输入：$A=(a_0,\ldots,a_{n-1})$,处理器数p

    输出：$S=\Sigma a_i$

```
Begin
    (1)S=0
    (2)for all Pi where 0≤i≤p-1 do
       (2.1) L=0
       (2.2) for j=i to n step p do
                L=L+aj
             end for
       (2.3) lock(S)
             S=S+L
       (2.4) unlock(S)
    end for
End
```

并行算法的通讯

- 通讯
  - 共享存储多处理器使用：global read(X,Y)和global write(X,Y)
  - 分布存储多计算机使用：send(X,i)和receive(Y,j)
- 通讯语句示例
  - 算法4.2 分布存储多计算机上矩阵向量乘算法

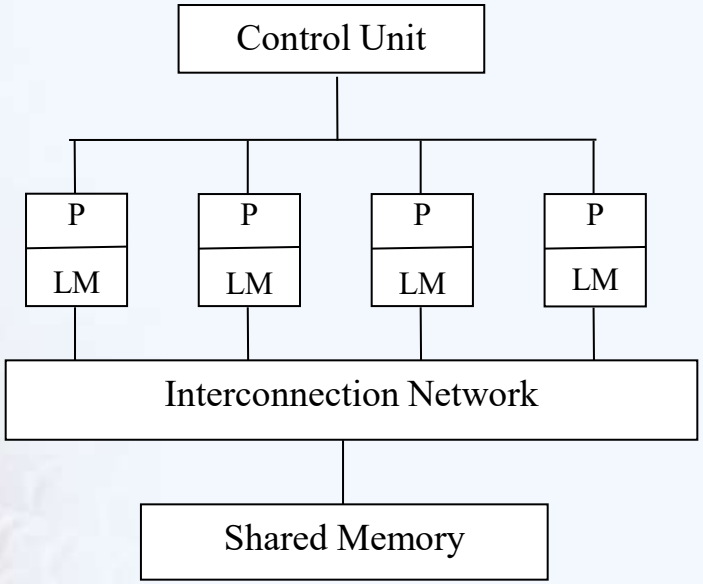
    输入：处理器数p, A划分为B=A[1..n,(i-1)r+1..ir],
    x划分为w=w[(i-1)r+1;ir]

    输出：P_1保存乘积AX

```
Begin
    (1) Compute z=Bw
    (2) if i=1 then y_i=0 else receive(y,left) endif
    (3) y=y+z
    (4) send(y,right)
    (5) if i=1 then receive(y,left)
End
```

PRAM模型

- 基本概念
  - 由Fortune和Wyllie1978年提出，又称SIMD-SM模型。有一个集中的共享存储器和一个指令控制器，通过SM的R/W交换数据，隐式同步计算。
- 结构图



PRAM模型

- 分类

  (1) PRAM-CRCW并发读并发写
  - CPRAM-CRCW(Common PRAM-CRCW)：仅允许写入相同数据
  - PPRAM-CRCW(Priority PRAM-CRCW)：仅允许优先级最高的处理器写入
  - APRAM-CRCW(Arbitrary PRAM-CRCW)：允许任意处理器自由写入

  (2)PRAM-CREW并发读互斥写

  (3)PRAM-EREW互斥读互斥写

- 计算能力比较
  - PRAM-CRCW是最强的计算模型，PRAM-EREW可logp倍模拟PRAM-CREW和PRAM-CRCW

$$T_{EREW} \geq T_{CREW} \geq T_{CRCW}$$

$$T_{EREW} = O(T_{CREW} \cdot \log p) = O(T_{CRCW} \cdot \log p)$$

PRAM模型

- 优点
  - 适合并行算法表示和复杂性分析，易于使用，隐藏了并行机的通讯、同步等细节。
- 缺点
  - 不适合MIMD并行机，忽略了SM的竞争、通讯延迟等因素

APRAM模型

- 基本概念
  - 又称分相（Phase）PRAM或MIMD-SM。每个处理器有其局部存储器、局部时钟、局部程序；无全局时钟，各处理器异步执行；处理器通过SM进行通讯；处理器间依赖关系，需在并行程序中显式地加入同步路障。
- 指令类型

(1)全局读　　(2)全局写

(3)局部操作　　(4)同步

APRAM模型

- 计算过程

  由同步障分开的全局相组成

	处理器 1	处理器 2	…	处理器 p
	read x_1	read x_3	…	read x_n
phase1	read x_2	*		*
	*	write to B		*
	write to A	write to C		write to D
同步障				
	read B	read A		read C
phase2	*	*		*
	write to B	write to D		
同步障				
	*	write to C		write to B
	read D			read A
				write to B
同步障				

APRAM模型

- 计算时间

设局部操作为单位时间；全局读/写平均时间为d，d随着处理器数目的增加而增加；同步路障时间为B=B(p)非降函数。

满足关系 $2 \le d \le B \le p$； $B = B(p) = O(d \log p)$ 或 $O(d \log p / \log d)$

令t_{ph}为全局相内各处理器执行时间最长者，则APRAM上的计算时间为

$$T = \sum t_{ph} + B \times \text{同步障次数}$$

- 优缺点

易编程和分析算法的复杂度，但与现实相差较远，其上并行算法非常有限，也不适合MIMD-DM模型。

BSP模型

- 基本概念
  - 由Valiant(1990)提出的，“块”同步模型，是一种异步MIMD-DM模型，支持消息传递系统，块内异步并行，块间显式同步。
- 模型参数
  - *p*：处理器数(带有存储器)
  - *l*：同步障时间(Barrier synchronization time)
  - *g*：带宽因子(time steps/packet)=1/bandwidth

BSP模型

- 计算过程

  由若干超级步组成，

  每个超级步计算模式如图

- 优缺点

  强调了计算和通讯的分离，提供了一个编程环境，易于程序复杂性分析。但需要显式同步机制，限制至多h条消息的传递等。

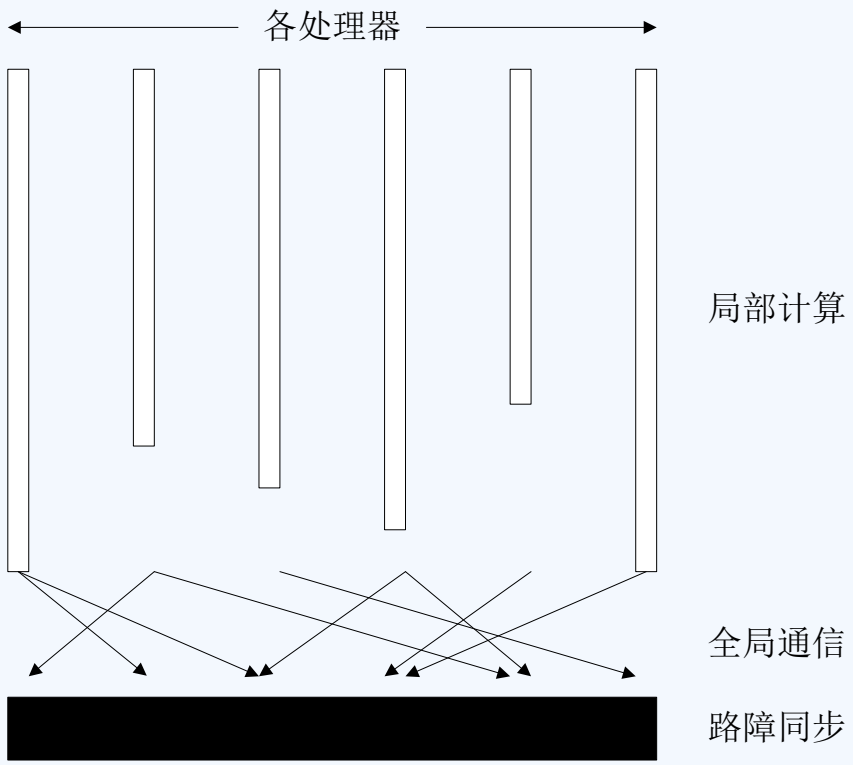


图4.3

LogP模型

- 基本概念
  - 由Culler(1993)年提出的，是一种分布存储的、点到点通讯的多处理机模型，其中通讯由一组参数描述，实行隐式同步。
- 模型参数
  - *L*：network latency
  - *o*：communication overhead
  - *g*：gap=1/bandwidth
  - *P*：#processors

注：*L*和*g*反映了通讯网络的容量

LogP模型

- 优缺点

  捕捉了MPC的通讯瓶颈，隐藏了并行机的网络拓扑、路由、协议，可以应用到共享存储、消息传递、数据并行的编程模型中；但难以进行算法描述、设计和分析。

- **BSP vs. LogP**
  - BSP→LogP：BSP块同步→BSP子集同步→BSP进程对同步＝LogP
  - BSP可以常数因子模拟LogP，LogP可以对数因子模拟BSP
  - BSP＝LogP+Barriers－Overhead
  - BSP提供了更方便的程设环境，LogP更好地利用了机器资源
  - BSP似乎更简单、方便和符合结构化编程

2.并行算法基本设计方法与技术

并行算法的基本设计方法

1 串行算法的直接并行化

2 从问题描述开始设计并行算法

3 借用已有算法求解新问题

串行算法的直接并行化

- 方法描述
  - 发掘和利用现有串行算法中的并行性，直接将串行算法改造为并行算法。
- 评注
  - 由串行算法直接并行化的方法是并行算法设计的最常用方法之一；
  - 不是所有的串行算法都可以直接并行化的；
  - 一个好的串行算法并不能并行化为一个好的并行算法；
  - 许多数值串行算法可以并行化为有效的数值并行算法。

从问题描述开始设计并行算法

- 方法描述
  - 从问题本身描述出发，不考虑相应的串行算法，设计一个全新的并行算法。
- 评注
  - 挖掘问题的固有特性与并行的关系；
  - 设计全新的并行算法是一个挑战性和创造性的工作；
  - 利用串的周期性的PRAM-CRCW算法是一个很好的范例；

借用已有算法求解新问题

- 方法描述
  - 找出求解问题和某个已解决问题之间的联系；
  - 改造或利用已知算法应用到求解问题上。
- 评注
  - 这是一项创造性的工作；
  - 使用矩阵乘法算法求解所有点对间最短路径是一个很好的范例。

并行算法的基本设计技术

1 划分设计技术
2 分治设计技术
3 平衡树设计技术
4 倍增设计技术
5 流水线设计技术

均匀划分技术

- **划分方法**

  n个元素A[1..n]分成p组，每组A[(i-1)n/p+1..in/p]，i=1~p

- **示例：** MIMD-SM模型上的PSRS排序

  begin

  (1)均匀划分：将n个元素A[1..n]均匀划分成p段，每个p_i处理A[(i-1)n/p+1..in/p]

  (2)局部排序：p_i调用串行排序算法对A[(i-1)n/p+1..in/p]排序

  (3)选取样本：p_i从其有序子序列A[(i-1)n/p+1..in/p]中选取p个样本元素

  (4)样本排序：用一台处理器对p^2个样本元素进行串行排序

  (5)选择主元：用一台处理器从排好序的样本序列中选取p-1个主元，并播送给其他p_i

  (6)主元划分：p_i按主元将有序段A[(i-1)n/p+1..in/p]划分成p段

  (7)全局交换：各处理器将其有序段按段号交换到对应的处理器中

  (8)归并排序：各处理器对接收到的元素进行归并排序

  end.

均匀划分技术

- 例6.1 PSRS排序过程。N=27，p=3，PSRS排序如下：

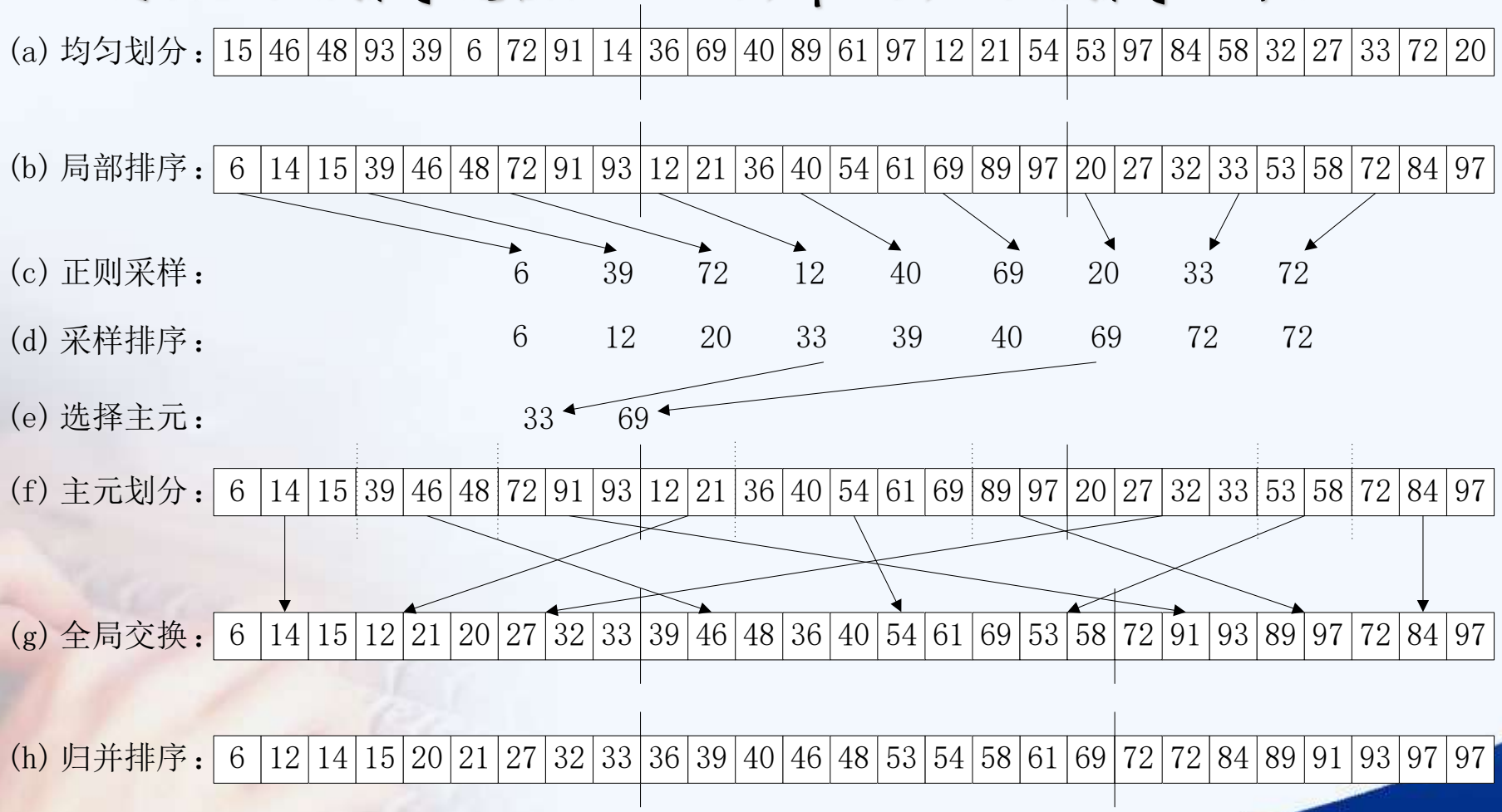


图6.1

方根划分技术

- **划分方法**

  n个元素A[1..n]分成A[(i-1)n^(1/2)+1..in^(1/2)]，i=1~n^(1/2)

- **示例：** SIMD-CREW模型上的 $k=\lfloor\sqrt{pq}\rfloor$ Valiant归并(1975年发表)

  //有序组A[1..p]、B[1..q], (假设p<=q), 处理器数 $k=\lfloor\sqrt{pq}\rfloor$

  begin

  (1)方根划分: A,B分别按 $i\lfloor\sqrt{p}\rfloor$ 和 $j\lfloor\sqrt{q}\rfloor$ 分成若干段（$i=1\sim\lfloor\sqrt{p}\rfloor$, $j=1\sim\lfloor\sqrt{q}\rfloor$）；

  (2)段间比较: A划分元与B划分元比较(至多 $\lfloor\sqrt{p}\rfloor\cdot\lfloor\sqrt{q}\rfloor$ 对)，
  确定A划分元应插入B中的区段；

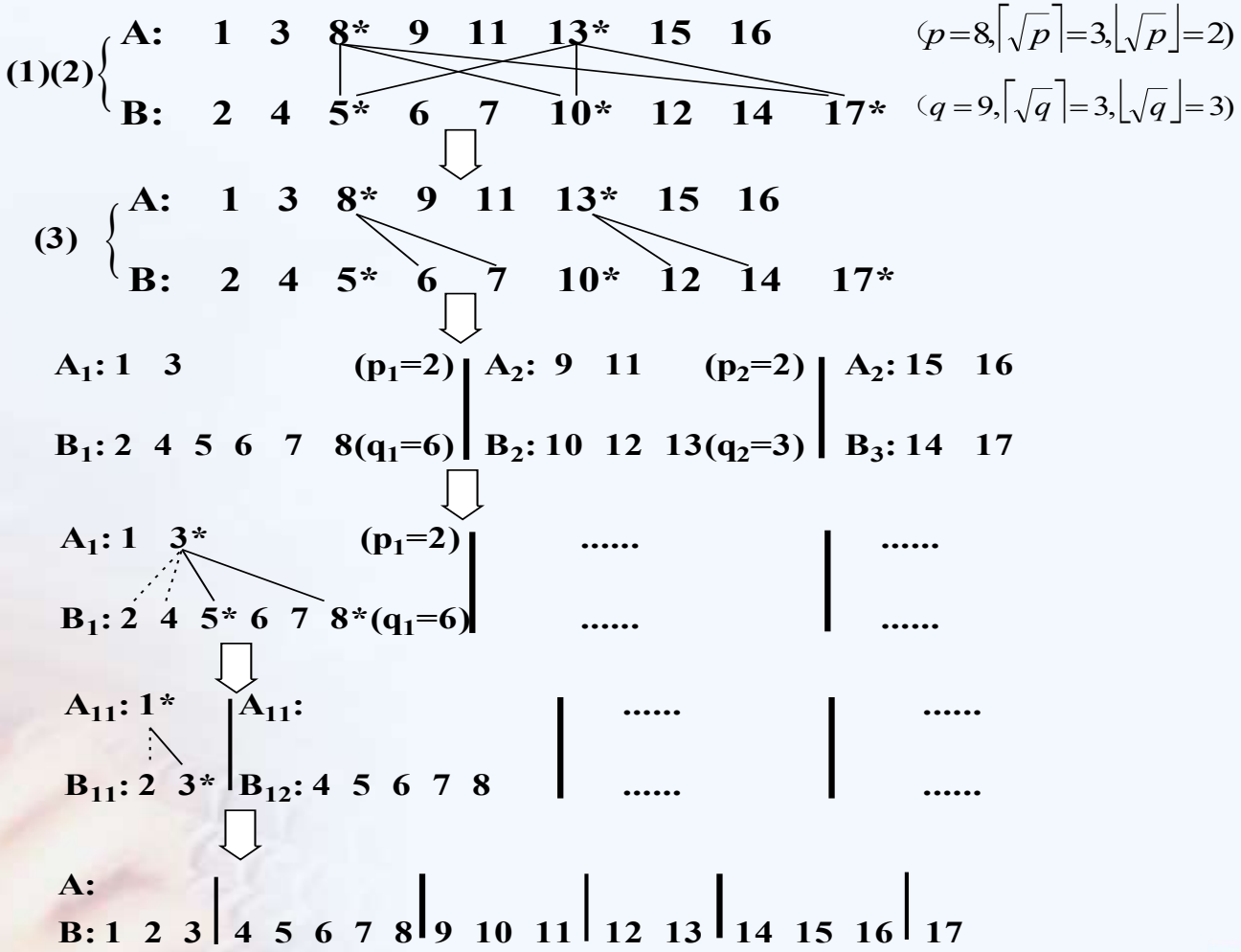
  (3)段内比较: A划分元与B相应段内元素进行比较，并插入适当的位置；

  (4)递归归并: B按插入的A划分元重新分段，与A相应段(A除去原划分元)构成了成对的段组，对每对段组递归执行(1)~(3)，直至A组为0时，递归结束；各组仍按 $k=\lfloor\sqrt{pq}\rfloor$ 分配处理器；

  end.

方根划分技术

- 示例: A={1,3,8,9,11,13,15,16},p=8; B={2,4,5,6,7,10,12,14,17},q=9



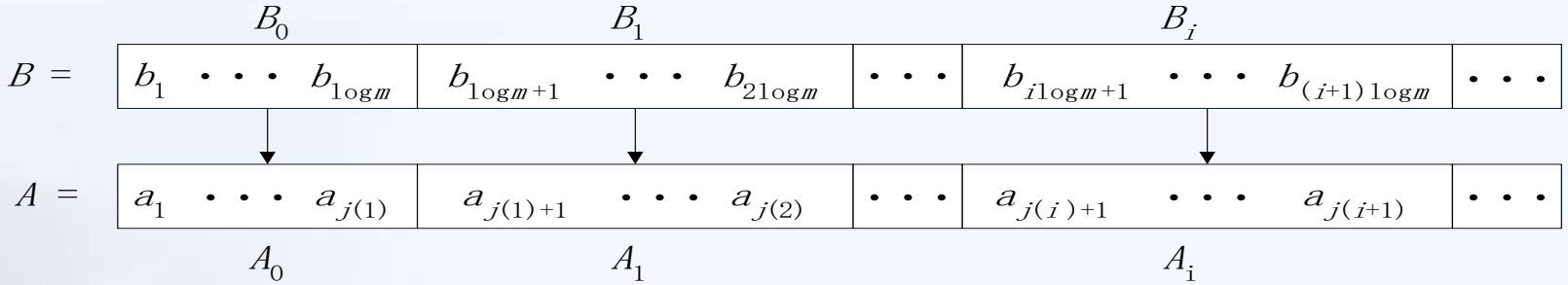
对数划分技术

- **划分方法**

  n个元素A[1..n]分成A[(i-1)logn+1..ilogn]，i=1~n/logn

- **示例：** PRAM-CREW上的对数划分并行归并排序

(1)归并过程：设有序组A[1..n]和B[1..m]



j[i]=rank(b_{ilogm}:A)为b_{ilogm}在A中的位序，即A中小于等于b_{ilogm}的元素个数

(2)例：A=(4,6,7,10,12,15,18,20), B=(3,9,16,21) n=8, m=4

=>logm=log4=2

=> j[1]=rank(b_{logm}:A)=rank(b_2:A)=rank(9:A)=3, j[2]=...=8

B_0: 3, 9　　B_1: 16, 21

A_0: 4, 6, 7　　A_1: 10, 12, 15, 18, 20

A和B归并化为(A_0, B_0)和(A_1, B_1)的归并

功能划分技术

- 划分方法

  n个元素A[1..n]分成等长的p组，每组满足某种特性。

- 示例：(m, n)选择问题(求出n个元素中前m个最小者)
  - 功能划分：要求每组元素个数必须大于m；
  - 算法：p148算法6.4

    输入：A=(a1,...,an)；输出：前m个最小者；

    Begin

    (1) 功能划分：将A划分成g=n/m组，每组含m个元素；

    (2) 局部排序：使用Batcher排序网络将各组并行进行排序；

    (3) 两两比较：将所排序的各组两两进行比较，从而形成MIN序列；

    (4) 排序-比较：对各个MIN序列，重复执行第(2)和第(3)步，直至选出m个最小者。

    End

功能划分技术

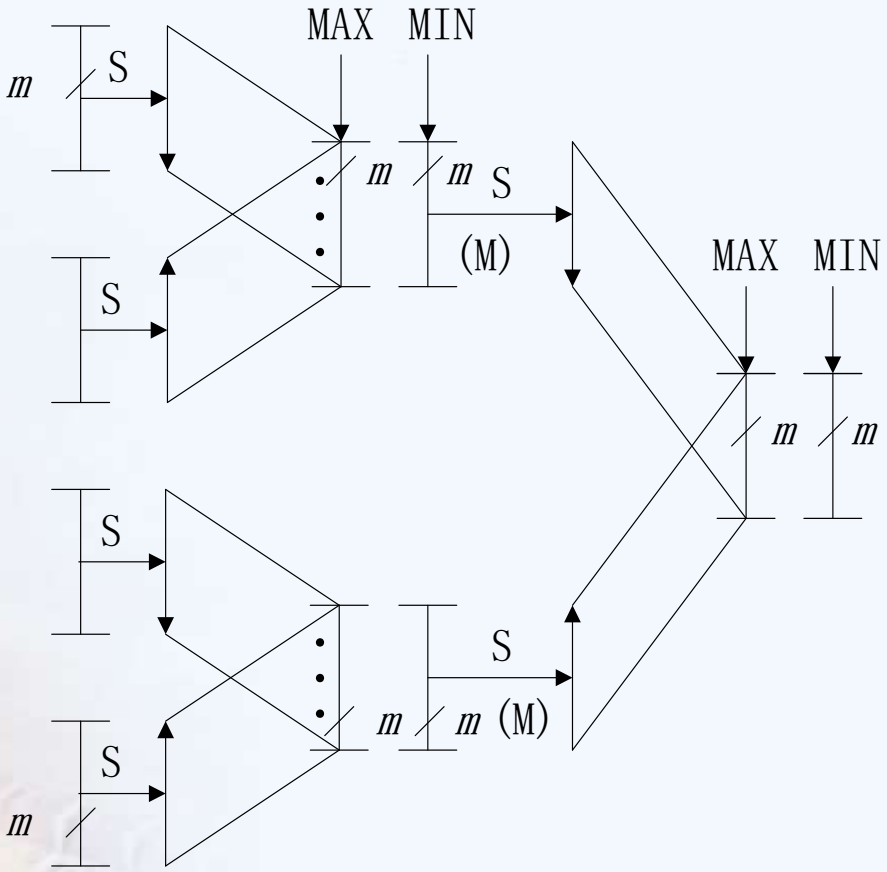


图6.3 (m-n)-选择过程

并行分治设计步骤

- 将输入划分成若干个规模相等的子问题；
- 同时(并行地)递归求解这些子问题；
- 并行地归并子问题的解，直至得到原问题的解。

双调归并网络

- 双调序列(先单调递增后单调递减，或反之)

  (1,3,5,7,8,6,4,2,0)

  (8,7,6,4,2,0,1,3,5)

  (1,2,3,4,5,6,7,8)

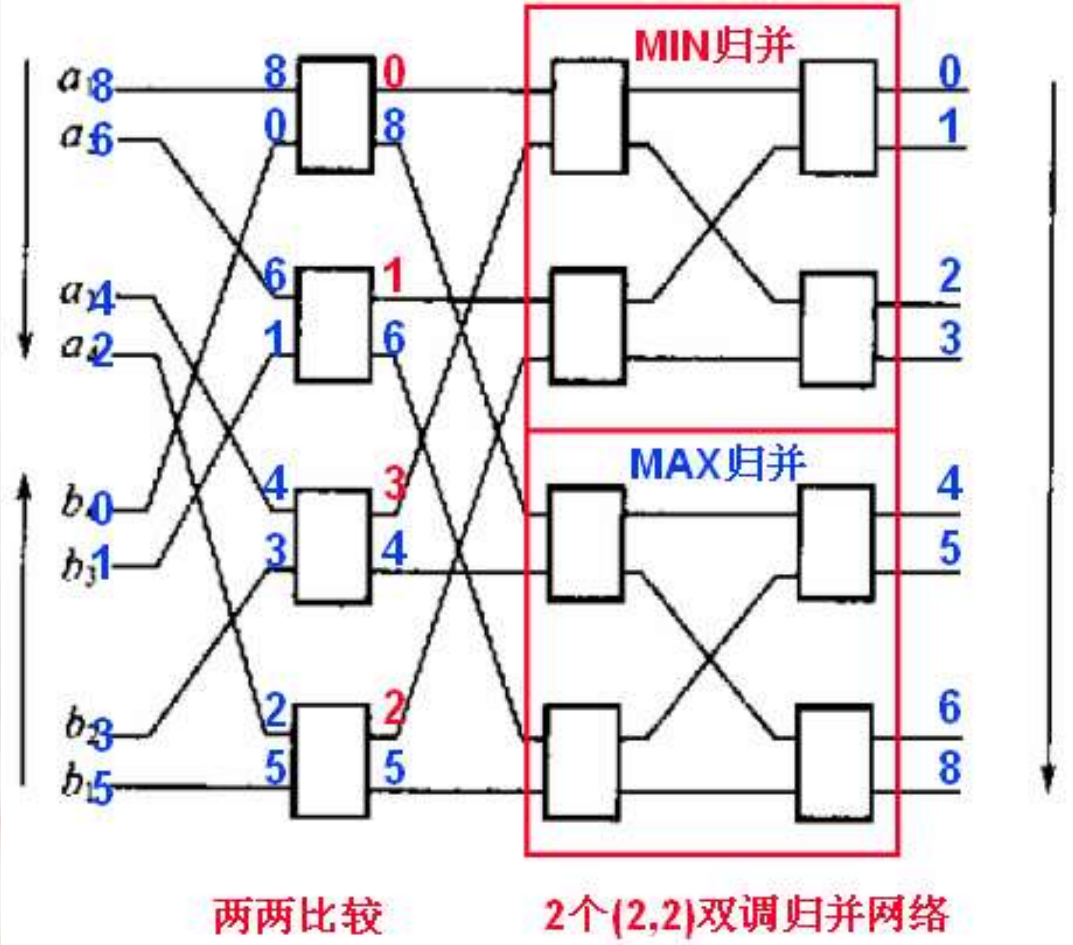
  以上都是双调序列

- Batcher定理

  给定双调序列$(x_0,x_1,\ldots,x_{n-1})$，对于$s_i=\min\{xi,xi+n/2\}$和$l_i=\max\{xi,xi+n/2\}$，则小序列$(s_0,s_1,\ldots,s_{n-1})$和大序列$(l_0,l_1,\ldots,l_{n-1})$仍是双调序列

双调归并网络

- (4,4)双调归并网络



双调归并网络

- Batcher双调归并算法

输入：双调序列$X=(x_0,x_1,\cdots,x_{n-1})$

输出：非降有序序列$Y=(y_0,y_1,\cdots,y_{n-1})$

```
Procedure BITONIC_MERG(x)
Begin
  (1)for i=0 to n/2-1 par-do
     (1.1) si=min{xi,xi+n/2}
     (1.2) li=max{xi,xi+n/2}
     end for
  (2)Recursive Call:
     (2.1)BITONIC_MERG(MIN=(s0,…,sn/2-1))
     (2.2)BITONIC_MERG(MIN=(l0,…, ln/2-1))
  (3)output sequence MIN followed by sequence MAX
End
```

平衡树设计技术

- 设计思想

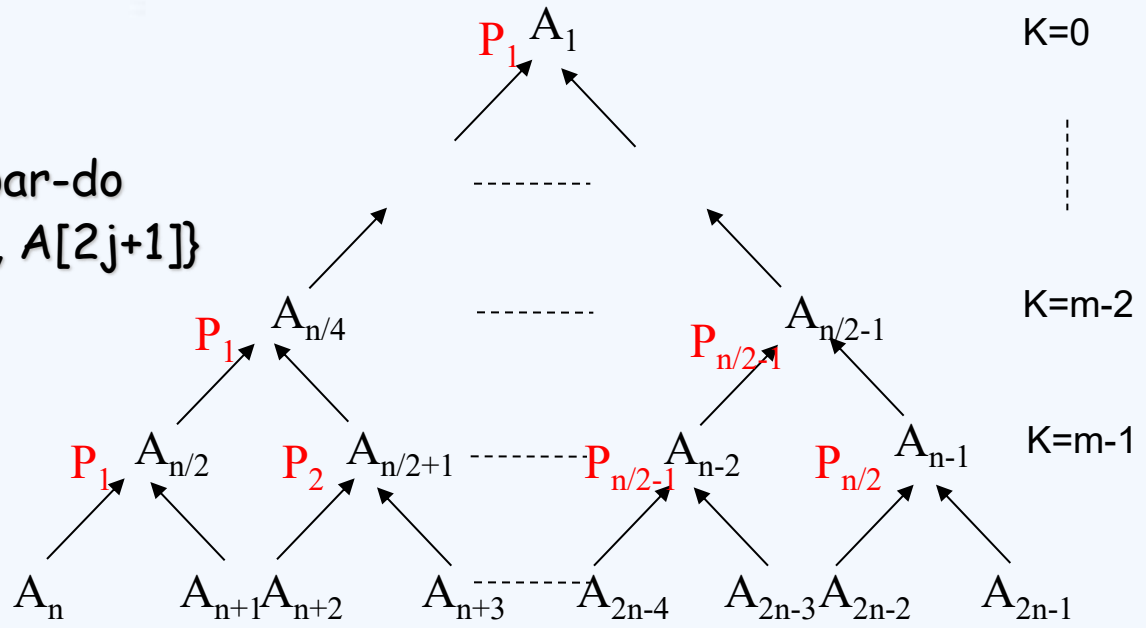
  以树的叶结点为输入，中间结点为处理结点，由叶向根或由根向叶逐层进行并行处理。

- 示例
  - 求最大值
  - 计算前缀和

求最大值

- 算法6.8：SIMD-TC(SM)上求最大值算法

```
Begin
  for k=m-1 to 0 do
     for j=2^k to 2^(k+1)-1 par-do
        A[j]=max{A[2j], A[2j+1]}
     end for
  end for
end
```



- 图示
- 时间分析

$t(n)=m\times O(1)=O(\log n)$

$p(n)=n/2$

计算前缀和

- 问题定义

  n个元素$\{x_1,x_2,\ldots,x_n\}$，前缀和是n个部分和：

  $S_i=x_1*x_2*\ldots*x_i$, $1\leqslant i\leqslant n$ 这里*可以是+或×

- 串行算法： $S_i=S_{i-1}*x_i$ 计算时间为O(n)

- 并行算法：SIMD-TC上非递归算法

  令$A[i]=x_i$, i=1~n,

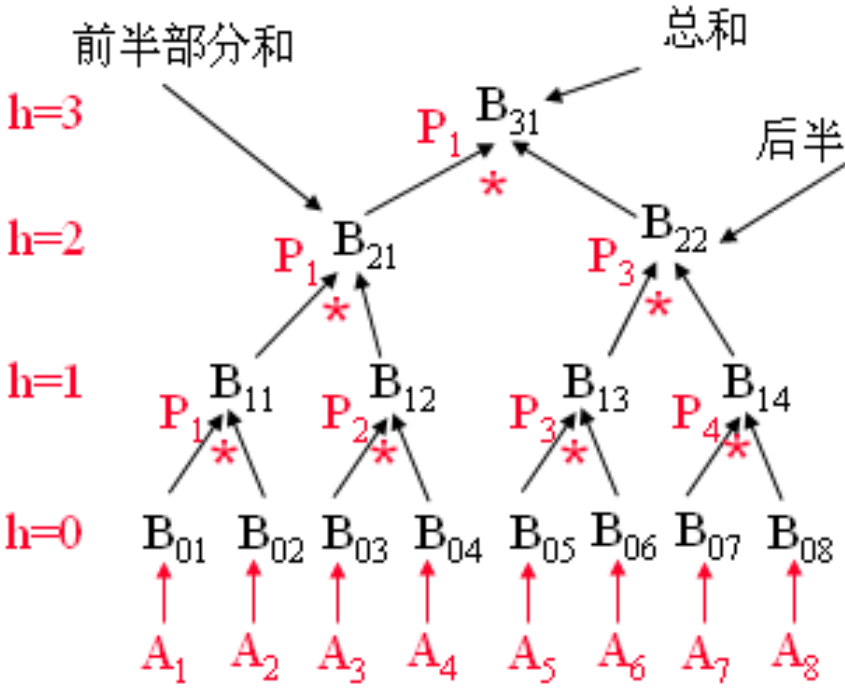
  B[h,j]和C[h,j]为辅助数组($h=0\sim\log n$, $j=1\sim n/2^h$)

  数组B记录由叶到根正向遍历树中各结点的信息(求和)

  数组C记录由根到叶反向遍历树中各结点的信息(播送前缀和)

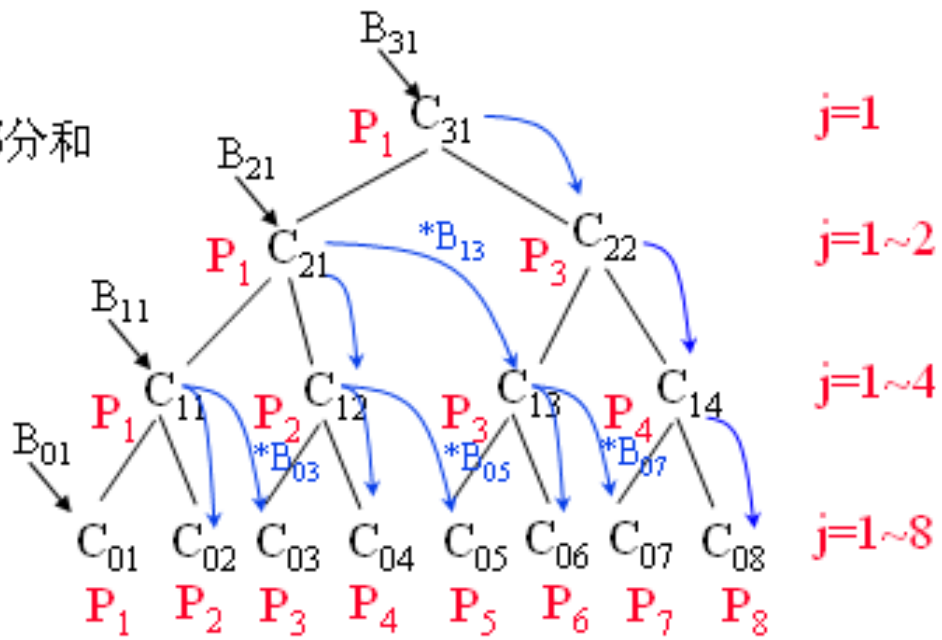
计算前缀和

- 例： n=8, p=8, C_{01}~C_{08}为前缀和



计算：B[h,j]=B[h-1,2j-1]*B[h-1,2j]

正向遍历(求和)



计算：C[h,1]=B[h,1] j=1

C[h,j]=C[h+1, j/2] j=even

C[h,j]=C[h+1,(j-1)/2]*B[h, j] j=odd>1

反向遍历(播送前缀和)

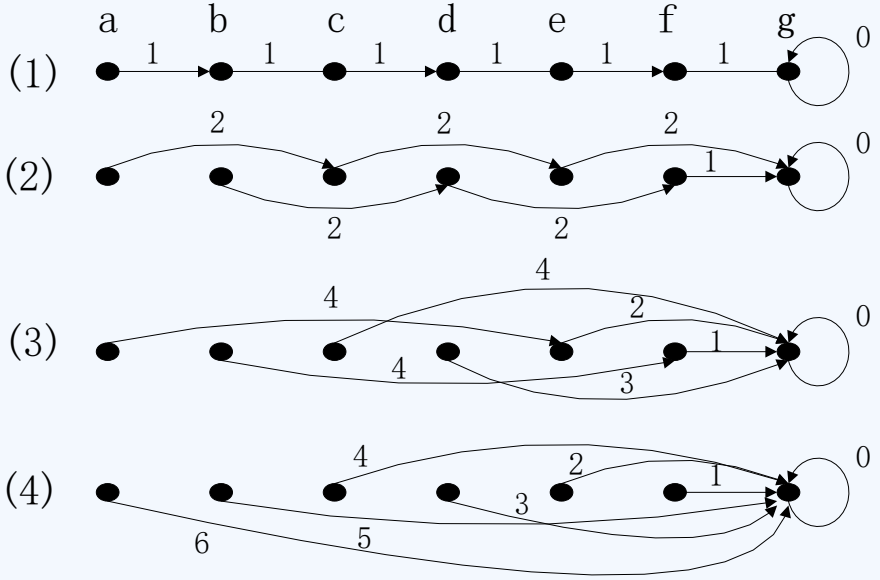
倍增设计技术

- 设计思想
  - 又称指针跳跃(pointer jumping)技术，特别适合于处理链表或有向树之类的数据结构；
  - 当递归调用时，所要处理数据之间的距离逐步加倍，经过k步后即可完成距离为2^k的所有数据的计算。
- 示例
  - 表序问题
  - 求森林的根

表序问题

- 问题描述

n个元素的列表L，求出每个元素在L中的次第号(秩或位序或rank(k))，rank(k)可视为元素k至表尾的距离；



- 示例： n=7

(1)p[a]=b, p[b]=c, p[c]=d, p[d]=e, p[e]=f, p[f]=g, p[g]=g
r[a]=r[b]=r[c]=r[d]=r[e]=r[f]=1, r[g]=0

(2)p[a]=c, p[b]=d, p[c]=e, p[d]=f, p[e]=p[f]=p[g]=g
r[a]=r[b]=r[c]=r[d]=r[e]=2, r[f]=1, r[g]=0

(3)p[a]=e, p[b]=f, p[c]=p[d]=p[e]=p[f]=p[g]=g
r[a]=4, r[b]=4, r[c]=4, r[d]=3, r[e]=2, r[f]=1, r[g]=0

(4)p[a]=p[b]=p[c]=p[d]=p[e]=p[f]=p[g]=g
r[a]=6, r[b]=5, r[c]=4, r[d]=3, r[e]=2, r[f]=1, r[g]=0

表序问题

- 算法：

(1)并行做：初始化p[k]和distance[k] //O(1)

(2)执行 $\lceil \log n \rceil$ 次 //O(logn)

(2.1)对k并行地做 //O(1)

如果k的后继不等于k的后继之后继，则

(i) distance[k]= distance[k]+ distance[p[k]]

(ii) p[k]=p[p[k]]

(2.2)对k并行地做

rank[k]=distance[k] //O(1)

运行时间：t(n)=O(logn) p(n)=n

求森林的根

- 问题描述

  一组有向树F中，如果<i, j>是F中的一条弧，则p[i]=j(即j是i的双亲)；若i为根，则p[i]=i。求每个结点j(j=1~n)的树根s[j].

- 示例

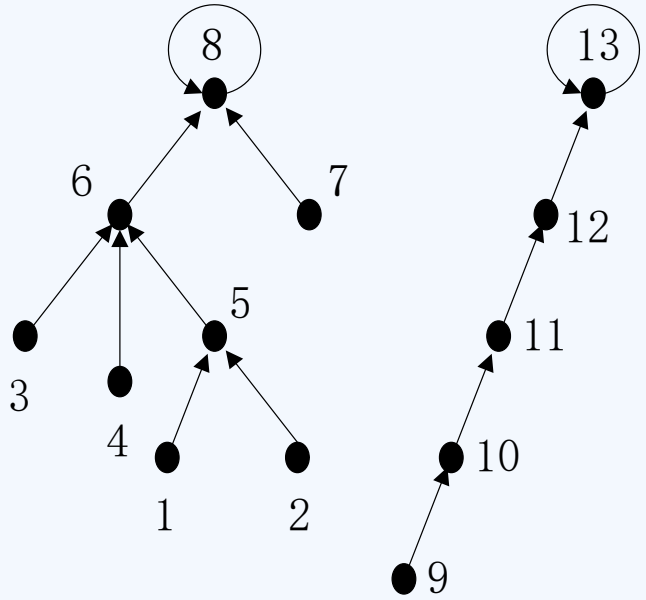
  初始时

  P[1]=p[2]=5 p[3]=p[4]=p[5]=6

  P[6]=p[7]=8 p[8]=8 P[9]=10

  p[10]=11 p[11]=12 p[12]=13 p[13]=13

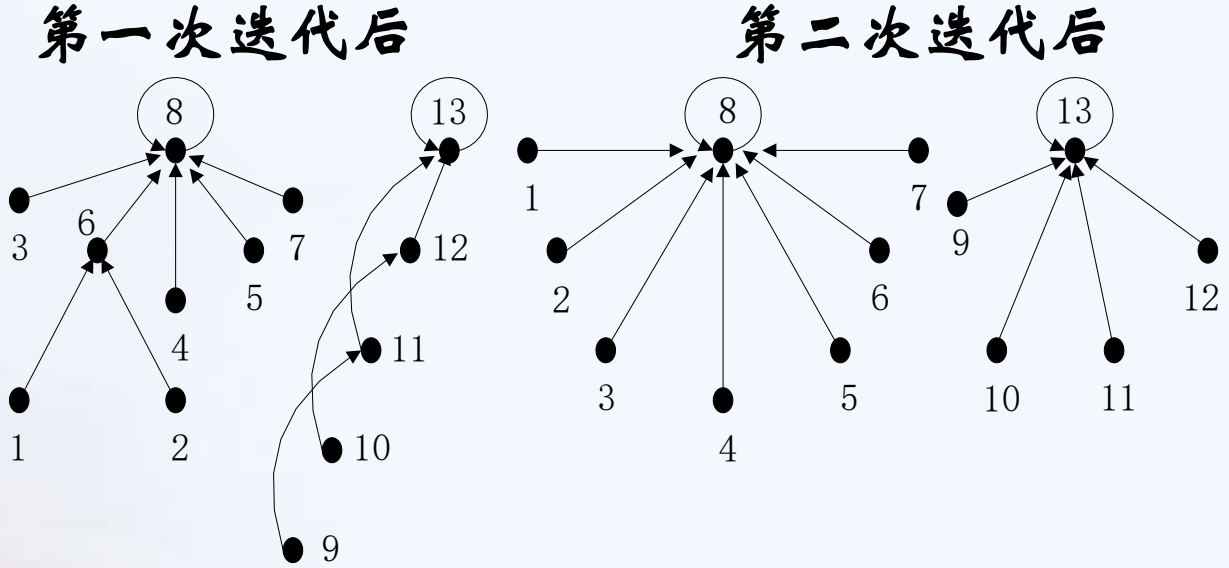
  s[i]=p[i]



(a)

求森林的根

- 示例



(b) (c)

- 运行时间：t(n)=O(logn) W(n)=O(nlogn)

流水线设计技术

- 设计思想
  - 将算法流程划分成p个前后衔接的任务片断，每个任务片断的输出作为下一个任务片断的输入；
  - 所有任务片断按同样的速率产生出结果。
- 评注
  - 流水线技术是一种广泛应用在并行处理中的技术；
  - 脉动算法(Systolic algorithm)是其中一种流水线技术；
  - 一种**数据流驱动的并行计算模式**，其核心思想是将计算过程设计成类似心脏“收缩-舒张”的节律性脉动：数据在规则的处理单元阵列中有节奏地流动，在每个时钟周期与局部存储的部分结果进行运算，最终在阵列边界输出完整结果。

5-point DFT的计算

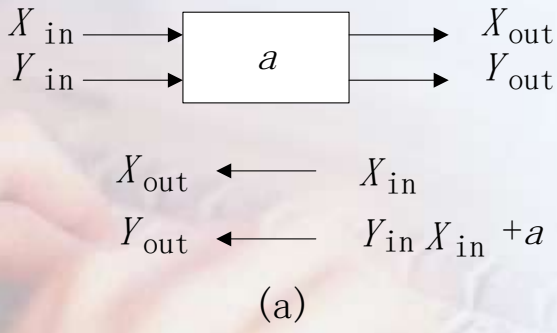
- 问题描述

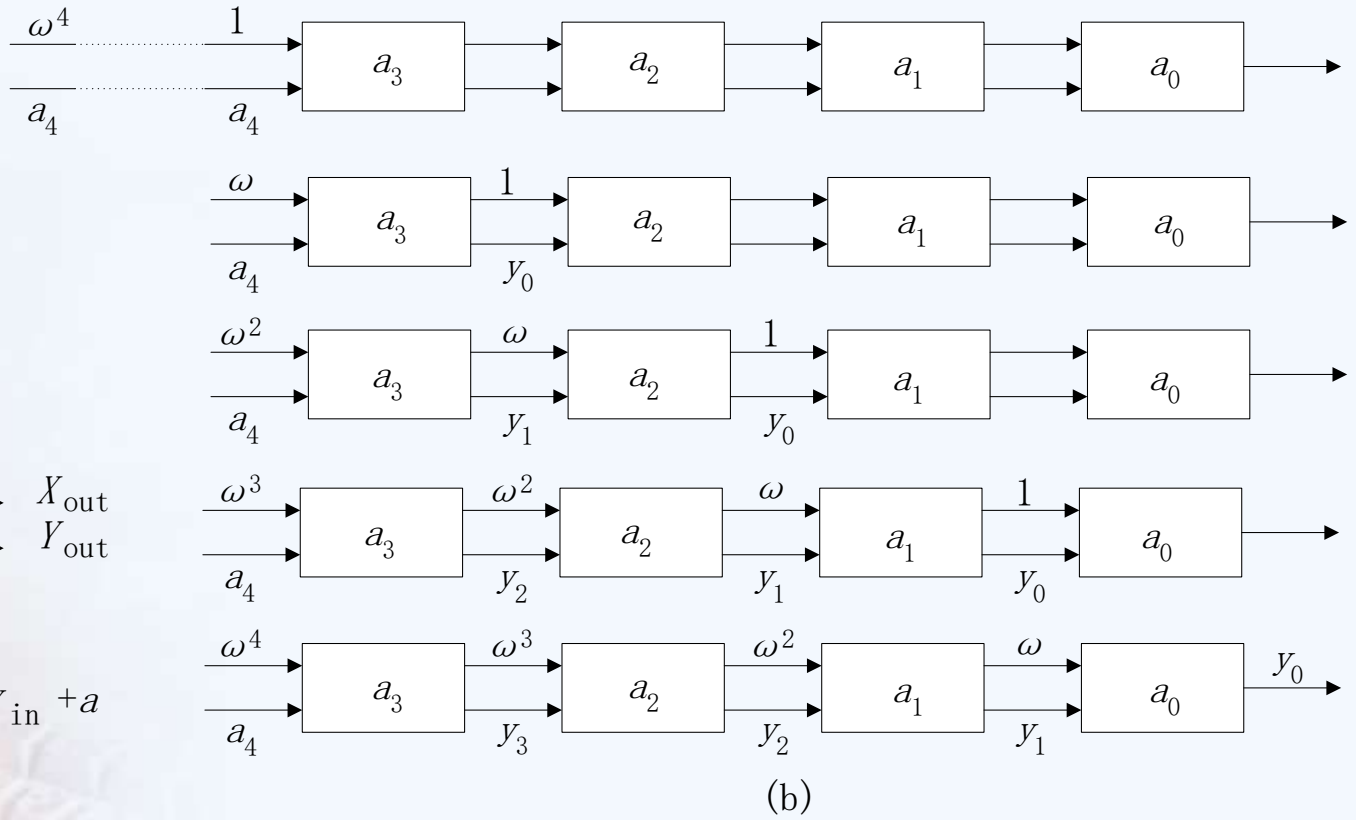
5-point DFT的计算。应用秦九韶(Horner)法则，

$$\begin{cases} y_0 = b_0 = a_4\omega^0 + a_3\omega^0 + a_2\omega^0 + a_1\omega^0 + a_0 \\ y_1 = b_1 = a_4\omega^4 + a_3\omega^3 + a_2\omega^2 + a_1\omega^1 + a_0 \\ y_2 = b_2 = a_4\omega^8 + a_3\omega^6 + a_2\omega^4 + a_1\omega^2 + a_0 \\ y_3 = b_3 = a_4\omega^{12} + a_3\omega^6 + a_2\omega^4 + a_1\omega^2 + a_0 \\ y_4 = b_4 = a_4\omega^{16} + a_3\omega^6 + a_2\omega^4 + a_1\omega^2 + a_0 \end{cases} \Rightarrow \begin{cases} y_0 = (((a_4\omega^0 + a_3)\omega^0 + a_2)\omega^0 + a_1)\omega^0 + a_0 \\ y_1 = (((a_4\omega^1 + a_3)\omega^1 + a_2)\omega^1 + a_1)\omega^1 + a_0 \\ y_2 = (((a_4\omega^2 + a_3)\omega^2 + a_2)\omega^2 + a_1)\omega^2 + a_0 \\ y_3 = (((a_4\omega^3 + a_3)\omega^3 + a_2)\omega^3 + a_1)\omega^3 + a_0 \\ y_4 = (((a_4\omega^4 + a_3)\omega^4 + a_2)\omega^4 + a_1)\omega^4 + a_0 \end{cases}$$

5-point DFT的计算

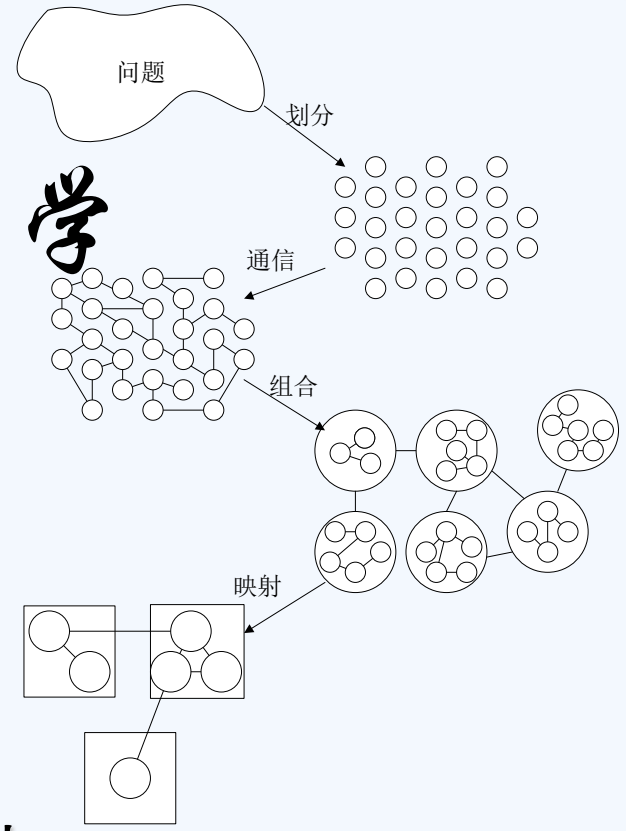
- 示例： p(n)=n-1, t(n)=2n-2=O(n)





3.并行算法的一般设计过程

PCAM设计方法学



- 设计并行算法的四个阶段
  - 划分(Partitioning)
  - 通讯(Communication)
  - 组合(Agglomeration)
  - 映射(Mapping)
- 划分：分解成小的任务，开拓并发性；
- 通讯：确定诸任务间的数据交换，监测划分的合理性；
- 组合：依据任务的局部性，组合成更大的任务；
- 映射：将每个任务分配到处理器上，提高算法的性能。

划分方法描述

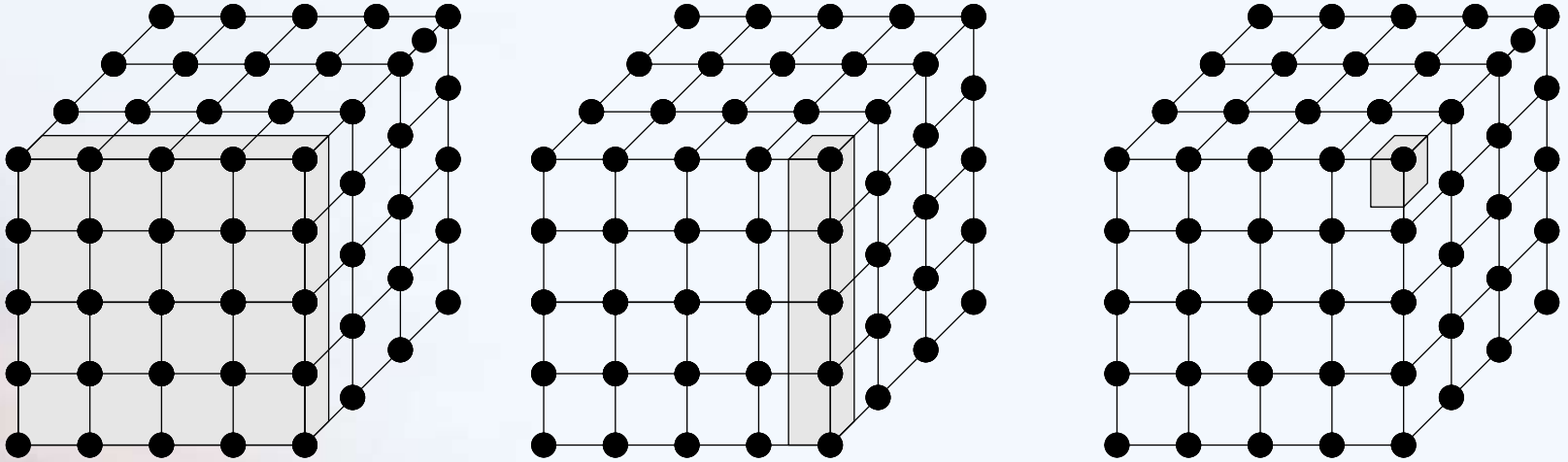
- 充分开拓算法的并发性和可扩放性；
- 先进行数据分解(称域分解)，再进行计算功能的分解(称功能分解)；
- 使数据集和计算集互不相交；
- 划分阶段忽略处理器数目和目标机器的体系结构；
- 能分为两类划分：
  - 域分解(domain decomposition)
  - 功能分解(functional decomposition)

域分解

- 划分的对象是数据，可以是算法的输入数据、中间处理数据和输出数据；
- 将数据分解成大致相等的小数据片；
- 划分时考虑数据上的相应操作；
- 如果一个任务需要别的任务中的数据，则会产生任务间的通讯；

域分解

- 示例：三维网格的域分解，各格点上计算都是重复的。下图是三种分解方法：

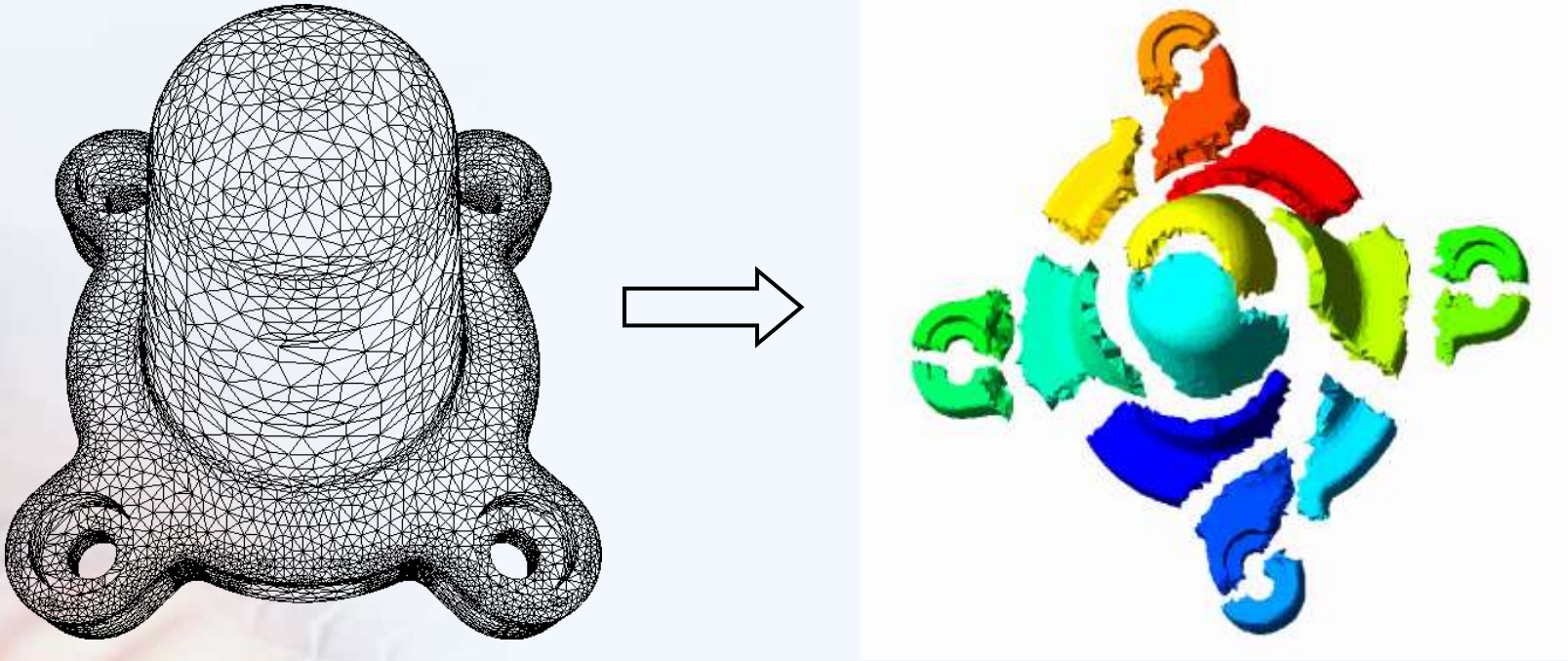


1-D　　2-D　　3-D

图7.2

域分解

- 不规则区域的分解示例：

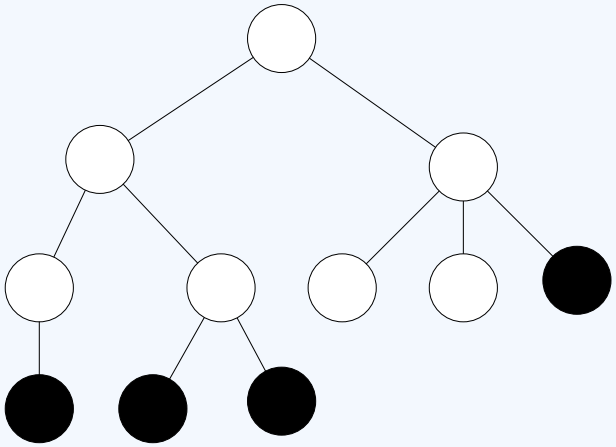


功能分解

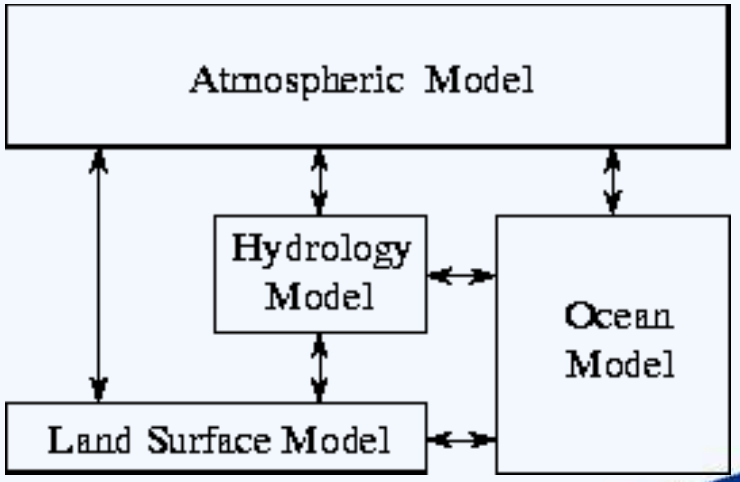
- 划分的对象是计算，将计算划分为不同的任务，其出发点不同于域分解；
- 划分后，研究不同任务所需的数据。如果这些数据不相交的，则划分是成功的；如果数据有相当的重叠，意味着要重新进行域分解和功能分解；
- 功能分解是一种更深层次的分解。

功能分解

- 示例1：搜索树



- 示例2：气候模型



划分判据

- 划分是否具有灵活性？
- 划分是否避免了冗余计算和存储？
- 划分任务尺寸是否大致相当？
- 任务数与问题尺寸是否成比例？
- 功能分解是一种更深层次的分解，是否合理？

通讯方法描述

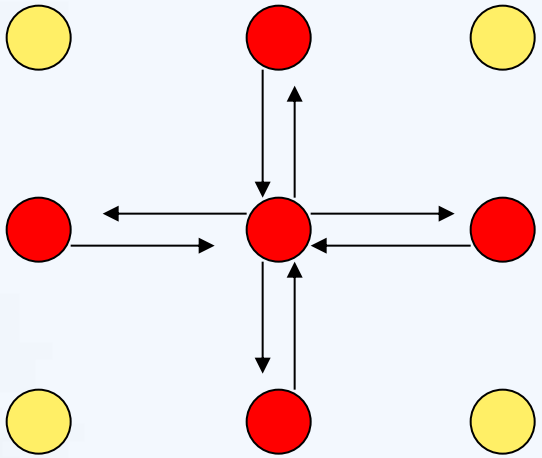
- 通讯是PCAM设计过程的重要阶段；
- 划分产生的诸任务，一般不能完全独立执行，需要在任务间进行数据交流；从而产生了通讯；
- 功能分解确定了诸任务之间的数据流；
- 诸任务是并发执行的，通讯则限制了这种并发性；

四种通讯模式

- 局部/全局通讯
- 结构化/非结构化通讯
- 静态/动态通讯
- 同步/异步通讯

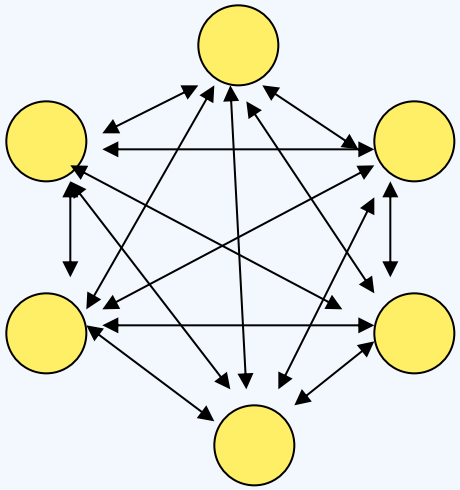
局部通讯

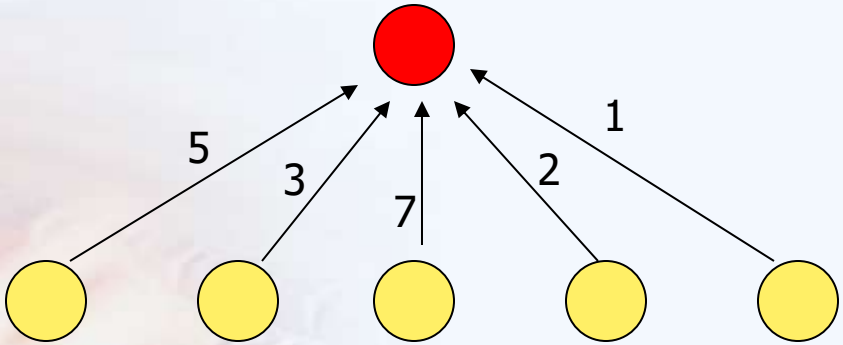
- 通讯限制在一个邻域内



全局通讯

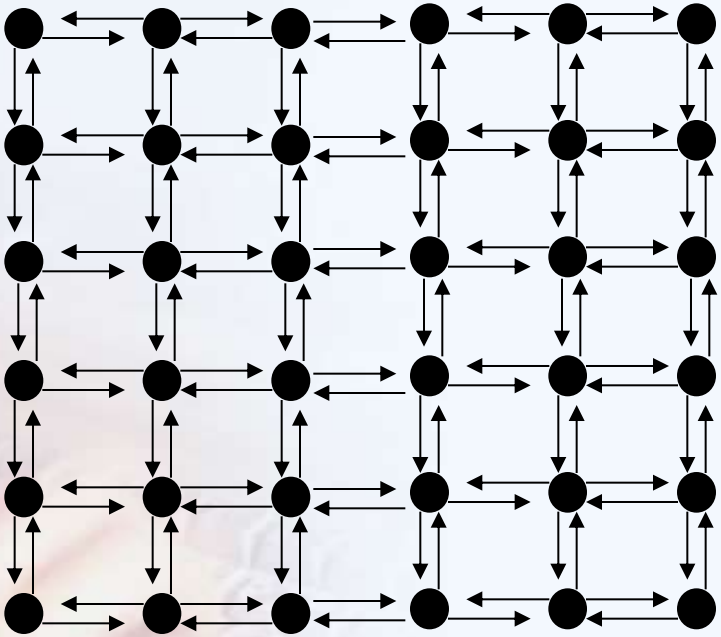
- 通讯非局部的
- 例如：
  - All to All
  - Master-Worker

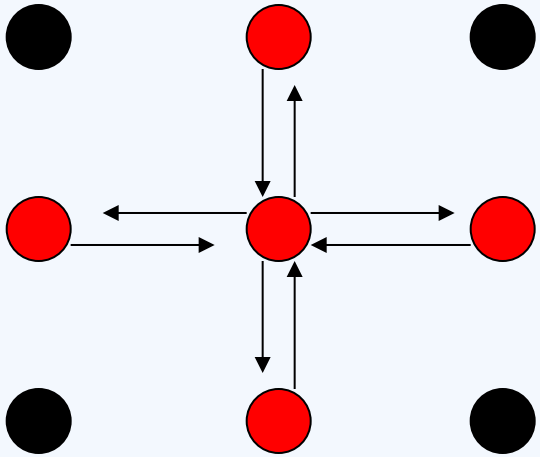




结构化通讯

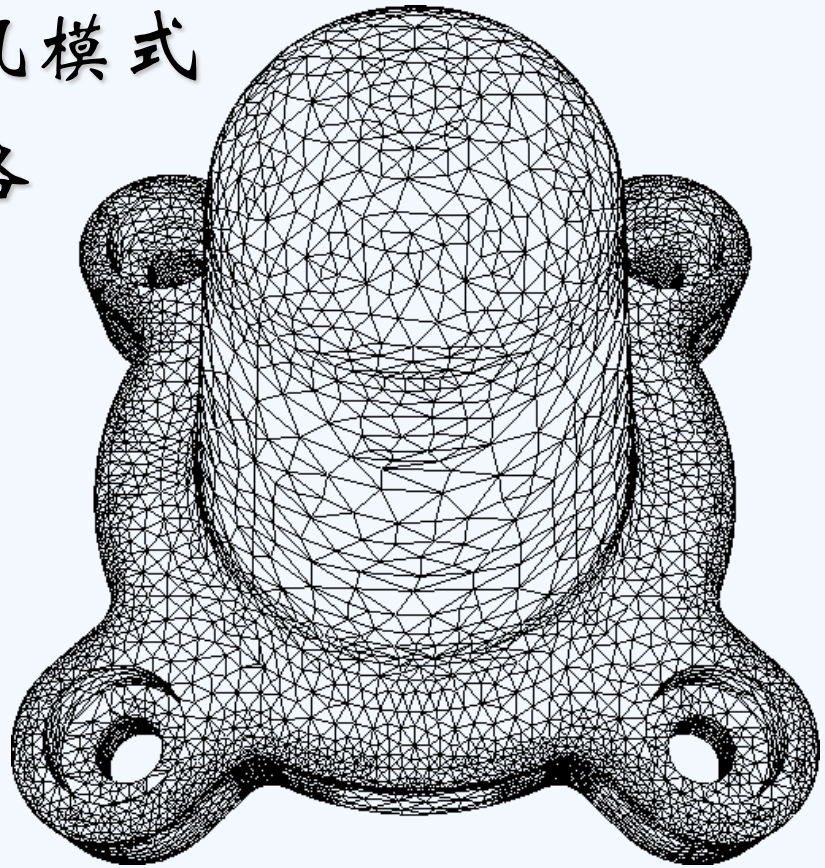
- 每个任务的通讯模式是相同的；
- 下面是否存在一个相同通讯模式？





非结构化通讯

- 没有一个统一的通讯模式
- 例如：无结构化网格



通讯判据

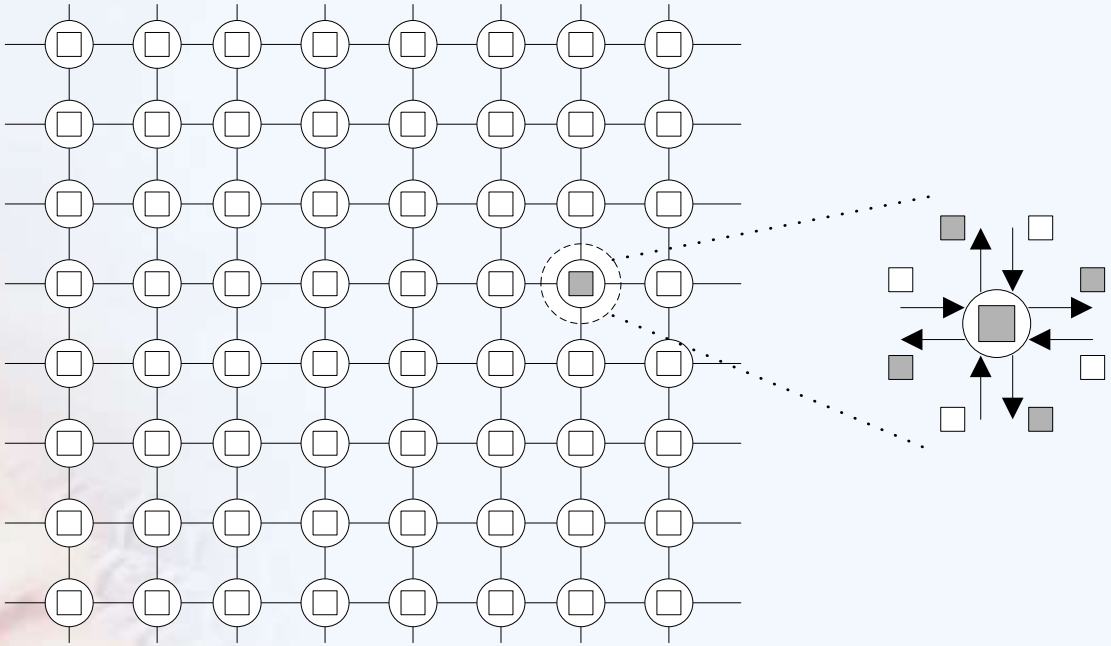
- 所有任务是否执行大致相当的通讯?
- 是否尽可能的局部通讯?
- 通讯操作是否能并行执行?
- 同步任务的计算能否并行执行?

组合方法描述

- 组合是由抽象到具体的过程，是将组合的任务能在一类并行机上有效的执行；
- 合并小尺寸任务，减少任务数。如果任务数恰好等于处理器数，则也完成了映射过程；
- 通过增加任务的粒度和重复计算，可以减少通讯成本；
- 保持映射和扩展的灵活性，降低软件工程成本；

表面-容积效应

- 通讯量与任务子集的表面成正比，计算量与任务子集的体积成正比；
- 增加重复计算有可能减少通讯量；

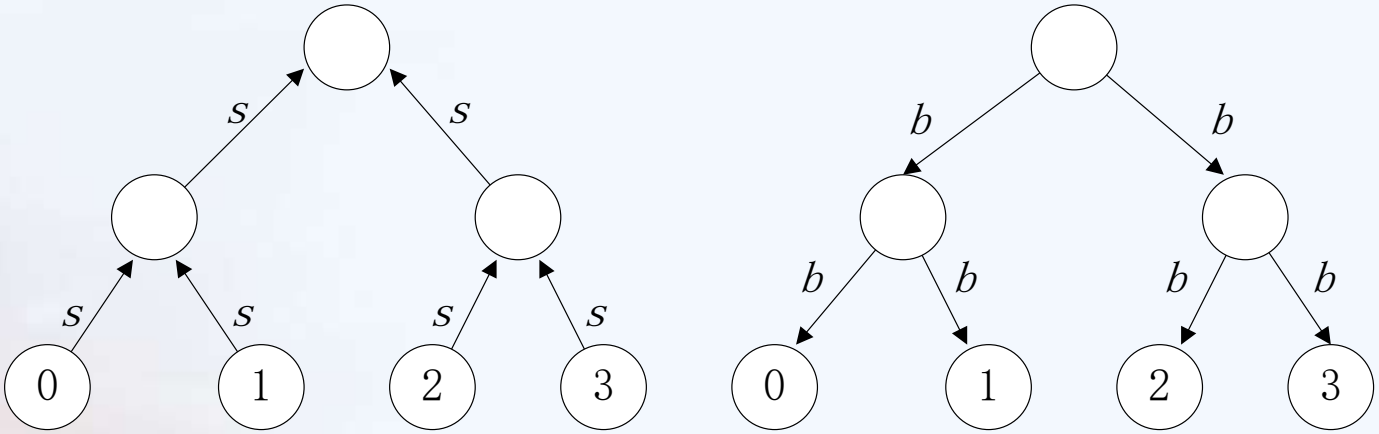


重复计算

- 重复计算减少通讯量，但增加了计算量，应保持恰当的平衡；
- 重复计算的目标应减少算法的总运算时间；

重复计算

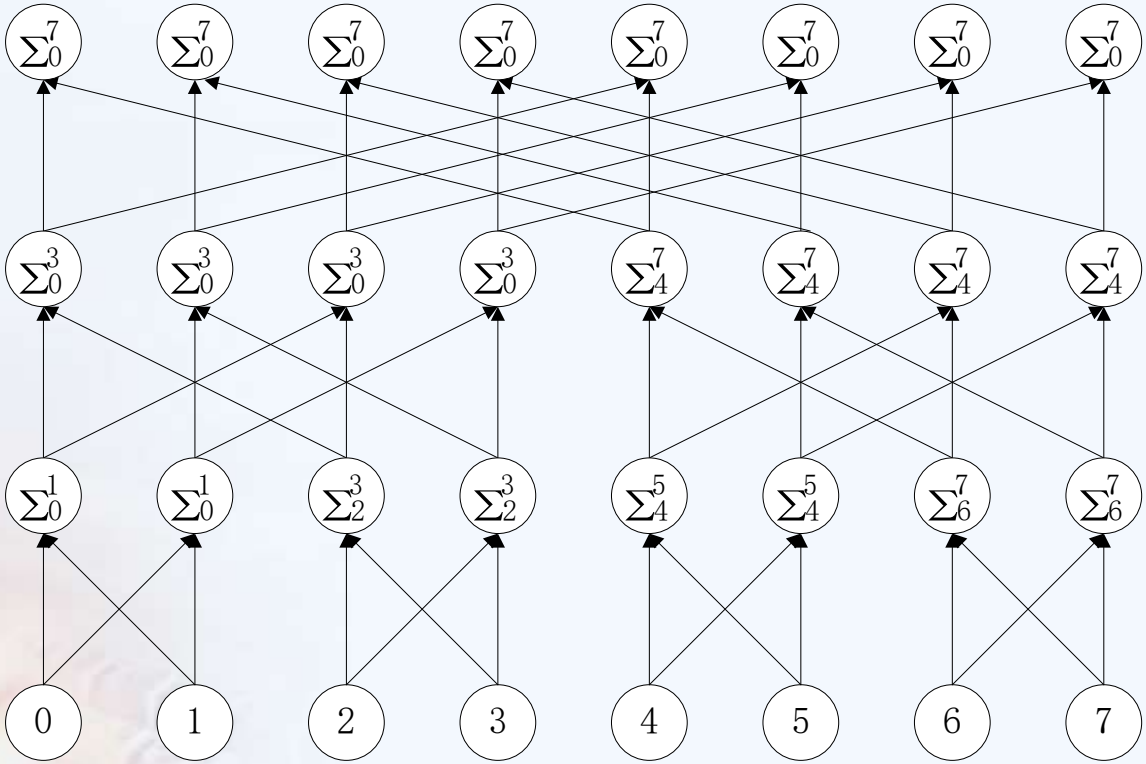
- 示例：二叉树上N个处理器求N个数的全和，要求每个处理器均保持全和。



二叉树上求和，共需2logN步

重复计算

- 示例：二叉树上N个处理器求N个数的全和，要求每个处理器均保持全和。



蝶式结构求和，使用了重复计算，共需logN步

组合判据

- 增加粒度是否减少了通讯成本?
- 重复计算是否已权衡了其得益?
- 是否保持了灵活性和可扩放性?
- 组合的任务数是否与问题尺寸成比例?
- 是否保持了类似的计算和通讯?
- 有没有减少并行执行的机会?

映射方法描述

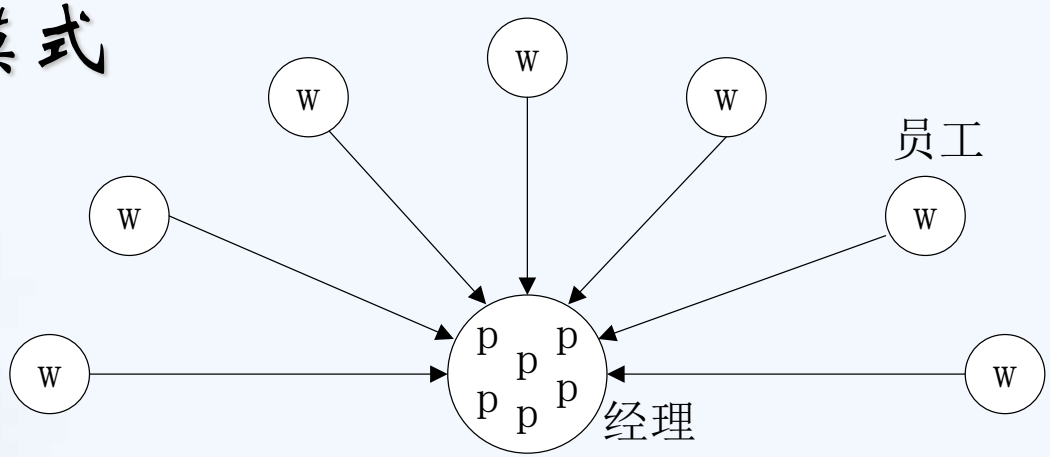
- 每个任务要映射到具体的处理器，定位到运行机器上；
- 任务数大于处理器数时，存在负载平衡和任务调度问题；
- 映射的目标：减少算法的执行时间
  - 并发的任务 → 不同的处理器
  - 任务之间存在高通讯的 → 同一处理器
- 映射实际是一种权衡，属于NP完全问题；

负载平衡算法

- 静态的：事先确定；
- 概率的：随机确定；
- 动态的：执行期间动态负载；
- 基于域分解的：
  - 递归对剖
  - 局部算法
  - 概率方法
  - 循环映射

任务调度算法

- 任务放在集中的或分散的任务池中，使用任务调度算法将池中的任务分配给特定的处理器。下面是两种常用调度模式：
- 经理/雇员模式



- 非集中模式

映射判据

- 采用集中式负载平衡方案，是否存在通讯瓶颈？
- 采用动态负载平衡方案，调度策略的成本如何？

小结

- 划分
  - 域分解和功能分解
- 通讯
  - 任务间的数据交换
- 组合
  - 任务的合并使得算法更有效
- 映射
  - 将任务分配到处理器，并保持负载平衡